Enabling Device-Initiated RDMA Communications in PTO-ISA on Ascend NPUs

Yuxin Yang[1], Zixiao Chen[2], Jieran Zhang[1], Chenyue Li[1], Tuo Dai[3], Bizhao Shi[3], Guojie Luo[1]
[1]School of Computer Science, Peking University [2]Shanghai Jiao Tong University [3]Independent Researcher
yxyang@pku.edu.cn, chen_zx@sjtu.edu.cn, jrzhang@pku.edu.cn, 2501112040@stu.pku.edu.cn
daitoto3@gmail.com, bizhao.shi@gmail.com, gluo@pku.edu.cn

***Abstract*—As models grow larger and inference workflows become more complex, distributed inference increasingly relies on moving key–value (KV) caches, activations, and mixture-of-experts (MoE) tokens across accelerators. Meeting low-latency requirements calls for device-initiated data movement across scale-up and scale-out systems, closely integrated with existing computation and memory-access programming. PTO-ISA provides a tile-based virtual instruction set with hardware-native interfaces for NPU computation, memory access, and synchronization, but extending this programming model across nodes requires an Ascend RDMA backend. We present a PTO-ISA communication extension that enables AI Vector (AIV) cores to drive RDMA over RoCE directly from device kernels. Compared with the Host-orchestrated HIXL and HCCL baselines, the extension combines device-side initiation, fine-grained transfer control, and direct composition with tile computation and memory operations. In the evaluated microbenchmarks, effective READ/WRITE throughput reaches 1.05× to 2.30× that of HIXL and 1.19× to 6.41× that of the HCCL ref. We further explore the extension's role in distributed inference through activation pipelines, KV-cache migration, and MoE dispatch and combine.**

***Index Terms*—PTO-ISA, Ascend NPUs, device-initiated RDMA**

I. Introduction

As models grow larger and inference becomes more complex, distributed inference increasingly relies on data movement across accelerators. Prefill–decode disaggregation migrates key–value (KV) caches [1], pipeline parallelism (PP) transfers activations [2]–[4], and mixture-of-experts (MoE) layers exchange routed tokens. These transfers place communication and its coordination with computation on the inference critical path. Phase separation enables specialized resource allocation but makes state transfer an explicit scheduling cost [5], [6].

Low-latency execution therefore calls for device-initiated communication across scale-up and scale-out systems, integrated with computation and memory-access programming. GPU networking and runtimes already expose device-side communication [8]–[11]. Tile-level control aligns transfers with data production and consumption [12], while explicit synchronization governs remote readiness and buffer reuse [13]. CoCoNeT similarly exposes computation and communication together to enable fusion and overlap [14].

PTO-ISA provides a tile-based virtual instruction set with hardware-native interfaces for NPU computation, memory access, and synchronization [15]. In the studied version, its Ascend backend uses SDMA for asynchronous PUT/GET; cross-node RDMA must preserve the same session/event semantics.

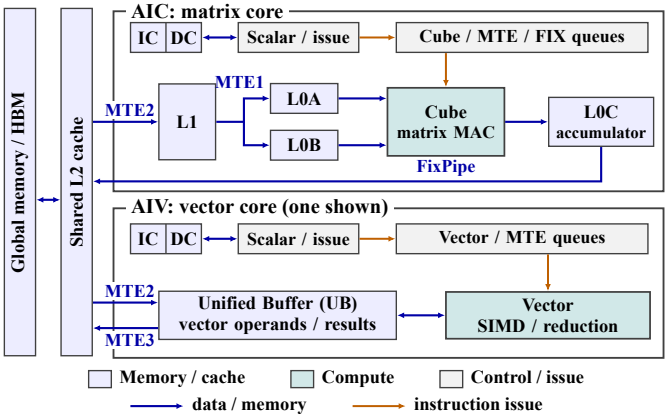


Fig. 1. Ascend NPU microarchitecture. [7]

We extend PTO-ISA with AI Vector (AIV)-driven RDMA over RoCE. AIV cores submit requests and process completions; NICs move payload between device memories. Compared with the Host-orchestrated HIXL and HCCL baselines [16], [17], the extension brings fine-grained transfer initiation and completion into the same device program as tile computation and memory operations. Batched regular-strided operations amortize queue submission, while ready/credit records distinguish local completion from remote consumption and slot reuse. The Host retains resource setup and kernel launch. READ/WRITE throughput reaches 1.05× to 2.30× that of HIXL and 1.19× to 6.41× that of the HCCL ref. We make the following contributions:

- We design and implement an AIV-driven RDMA backend for PTO-ISA, mapping its session/event interface to device-owned submission and completion queues while retaining Host-side resource setup.
- We develop batched and merged transfers for regular-strided data, and ready/credit protocols that connect transfer completion to remote consumption and safe buffer reuse in tile programs.
- We evaluate submission overhead and concurrent progress against HIXL and an HCCL reference. Activation pipelines, KV-cache migration, and MoE dispatch and combine illustrate how communication granularity affects application progress.

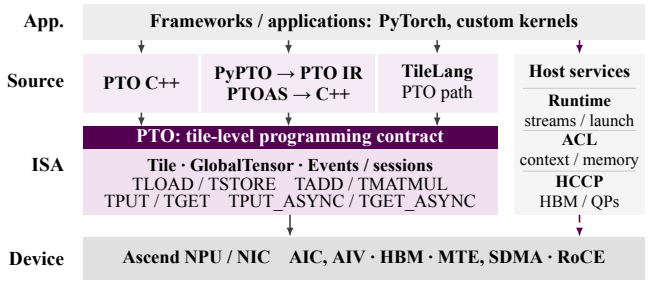


Fig. 2. PTO-related software stack. The ISA connects frontend kernels to device computation and communication; Host services register memory, set up communication resources, and launch kernels.

II. Background

A. Ascend NPU

As shown in Fig. 1, Ascend A2/A3 NPUs separate matrix and vector computation into AI Cube (AIC) and AI Vector (AIV) cores, each with independent scalar control [7]. The Cube unit uses L1 to stage operands, L0A/L0B to supply them, and L0C to accumulate results; the Vector unit operates on the Unified Buffer (UB). Global memory (GM) resides in HBM behind a shared L2 cache and carries data between AIC and AIV. Memory transfer engines (MTEs) move data explicitly: MTE2 loads GM into local buffers, MTE1 feeds L0A/L0B from L1, and MTE3 stores UB data to GM; FixPipe exports Cube results. Compute and transfer pipelines run asynchronously; events order buffer production, consumption, and reuse, while inter-core synchronization orders dependencies through GM.

B. PTO-ISA

PTO-ISA is Ascend CANN's tile-oriented virtual instruction set for Parallel Tile Operation (PTO) [15]. It uses typed on-chip tiles, `GlobalTensor` memory views, and explicit events to express computation, data movement, and pipeline dependencies. Fig. 2 shows how PTO C++ and frontend-generated kernels map to target-specific Tile Library implementations.

PTO-ISA Comm supports inter-NPU data movement, signal synchronization, and collective operations [15]. Synchronous `TPUT`/`TGET` perform remote writes and reads through MTE2–UB–MTE3. Asynchronous `TPUT_ASYNC`/`TGET_ASYNC` select a transfer engine, hold communication state in a session, and return an event for completion checks. `TNOTIFY`, `TWAIT`, and `TTEST` provide remote notification and signal checks; collective instructions cover gather, scatter, reduce, and broadcast.

III. Design and Implementation

The extension separates transfer semantics, queue execution, and application readiness. Sessions bind byte-range operations to device-owned queues; regular batches amortize submission; ready and credit govern remote consumption and slot reuse.

A. RDMA Operations and Sessions

A WRITE copies local data to a registered peer range; a READ fetches peer data into local memory. Both retain PTO-ISA's asynchronous event bound to a session and target completion sequence. Listing 1 shows submission and local completion; application readiness is established separately.

```
1 BuildAsyncSession<DmaEngine::RDMA>(
2     scratch, workspace, peer, rank, session, 0);
3 auto event = TPUT_ASYNC<DmaEngine::RDMA>(
4     remote_tensor, local_tensor, session);
5 auto status = rdma::WaitEventStatus(
6     event.handle, session);
```

Listing 1. Asynchronous RDMA WRITE and local completion.

Before launch, Host ACL/HCCP services register HBM, create queue pairs (QPs), and export queue buffers and doorbells. TCP exchanges connection and region metadata. A versioned workspace records these resources; independent region bases and protection keys avoid a symmetric-heap requirement. Pinned SHMEM source supplies the queue-format and HCCP ABI reference [18]. Operations validate both registered ranges, and callers preserve buffers until completion. The workspace remains valid across kernel launches, so steady-state transfers reuse registered memory and connected QPs. Successful shutdown drains outstanding operations before releasing owned resources; concurrent deregistration and reconnection are outside this prototype's interface.

Registration bounds also constrain strided operations. For region base b, length L, address a, and transfer size n, validation requires $a \geq b$, $a - b \leq L$, and $0 < n \leq L - (a - b)$ at both endpoints. This subtraction-based check avoids overflow in the end address. Hoisting these checks is valid only when the complete regular range remains inside its registration; it does not extend that registration's lifetime.

B. Ordered Device Execution

Fig. 3(a) follows a WRITE. The AIV encodes addresses, length, keys, opcode, and owner state in UB scratch. MTE3 publishes the WQE to the send queue (SQ), and an event completes that store before the doorbell notifies the NIC. The NIC moves payload directly between HBMs. READ reverses payload direction while retaining the same submission path.

One AIV owns each QP. With published count h, reclaimed count c, and depth D, it maintains $0 \leq h - c < D$; ring indices wrap modulo D and owner bits distinguish laps. Each WQE is signaled. MTE2 loads completion queue entries (CQEs), allowing the AIV to check owner, status, and QP number before reclaiming capacity. These loads avoid stale scalar-cache observations. An event targeting sequence t completes successfully when $c \geq t$; independent work may precede its wait. Multi-peer programs use one session per peer. Waits have timeouts, terminal errors reject further submissions, and failed transfers may leave partial remote writes.

The tested SQ and CQ each contain 8,192 entries, with 64-byte WQEs and 32-byte CQEs. Nonblocking tests process a bounded number of CQEs before returning control to computation. Physical ring wrap preserves the logical sequence; exhaustion of its 32-bit counter requires a new session.

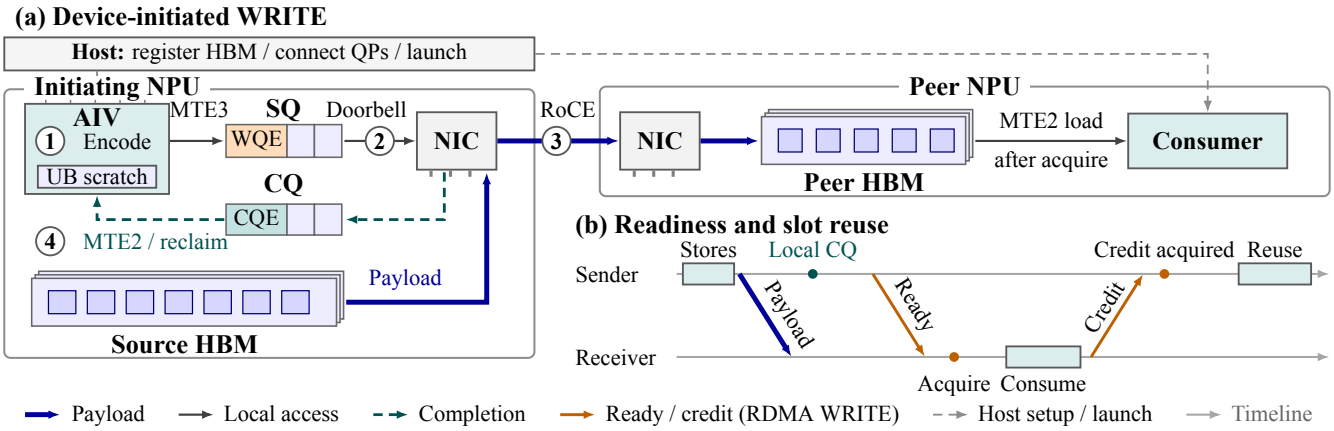


Fig. 3. Device-initiated RDMA WRITE in the PTO-ISA extension. (a) Encode a WQE (1), publish through MTE3 and ring the doorbell (2), transfer payload through NICs (3), and load CQEs through MTE2 (4). (b) Timeline of data readiness, consumption, and slot reuse.

C. Amortizing Regular-Strided Submission

Regular-strided operations add a segment count, independent endpoint strides, and publication options to the segment tensors. The prototype exposes `TPUT_STRIDED_ASYNC` for WRITE and `TGET_STRIDED_ASYNC` for READ. Arbitrary page lists use separate planners. *Base* validates and publishes each segment. *Hoist* checks regular ranges and reads queue indices once. *Batch* additionally publishes up to 256 WQEs per doorbell. *Merge* combines segments only when both endpoint sequences are contiguous within their registrations and fit the hardware length limit. *Combined* enables Batch and Merge.

These transformations reduce validation, publications, or WQE count without changing completion semantics. Remaining descriptors are written separately and signaled; a batch reserves capacity, preserves order, and returns an event for its final WQE. Validation does not make network errors atomic. Batching therefore amortizes doorbells even when gaps prevent merging, whereas merging also removes descriptor construction and completion work. The two optimizations address different costs and need not provide equal benefits for a given payload size. With gaps, a batch retains one WQE per segment and shares the doorbell. Contiguous source and destination segments can instead merge into fewer WQEs; the returned event tracks the resulting queue sequence.

D. Remote Readiness and Buffer Reuse

Batch completion still leaves the remote consumer's dependency unresolved. Fig. 3(b) therefore orders producer stores, payload WRITE completion, ready publication, consumer acquire, consumption, and credit return. Ready and credit are ordinary RDMA WRITEs to separate 64-byte records containing sixteen copies of a nonwrapping generation. An MTE2 acquire requires every word to match, assuming neither an atomic 64-byte write nor a remote-cache acquire at the initiating CQ. Polling reloads the full record; mixed generations retry, while an unexpected future generation returns an error. A producer reuses a slot only after acquiring its credit; withheld credit produces a bounded timeout and prevents the next write to that slot. Ready authorizes consumption of one generation; only its returned credit authorizes overwriting that slot with the next generation.

Activation runs this loop persistently. KV places short acquire kernels on each layer's compute stream, releasing the AIV before computation. MoE uses ready acquires between Host-scheduled dispatch, grouped GEMM, and return phases. The evaluation first isolates transfer costs and concurrent progress, then examines how transfer granularity and ready/credit dependencies govern application progress.

IV. Evaluation

A. Experimental Setup

We use two servers, each with eight Ascend 910C cards exposed as sixteen logical devices, connected through RoCE. The software stack uses driver 25.2.1 and CANN 9.0.0. We compare our extension, labeled PTO, with the Host-orchestrated HIXL sync/async and HCCL Send/Recv implementations from the installed CANN libraries [16], [17].

For primitive and bidirectional measurements, PTO/HIXL timing ends at initiator completion; HCCL additionally includes receiver completion and packing/unpacking when needed. We therefore report HCCL as a reference under different timing boundaries. Application cases measure complete communication tasks, with their timing scopes specified below.

B. Effective READ/WRITE Throughput

Fig. 4 pairs contiguous and strided transfers from 64 KB to 64 MB. Three counterparts supplement the confirmation data with identical binaries. PTO uses Combined and HIXL uses sync. HCCL uses BatchSendRecv for contiguous input and packing for strided input.

Under the timing definitions in Sec. IV-A, effective READ-/WRITE throughput reaches 1.05× to 2.30× that of HIXL and 1.19× to 6.41× that of the HCCL reference. Gaps are largest for small or fragmented transfers and narrow for larger contiguous payloads. HCCL packing reduces per-extent communication but adds local copies; PTO batches strided descriptors directly. The results reflect both submission and layout handling. The following ablation separates the mechanisms behind these costs.

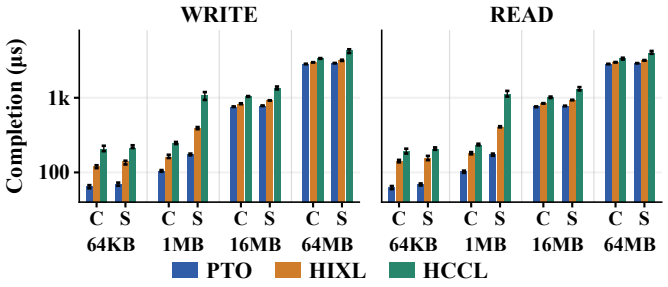


Fig. 4. READ/WRITE latency. C/S: contiguous/strided; HCCL: reference.

TABLE I
PTO TRANSFER LATENCY (μS) UNDER FOUR LOWERING VARIANTS. S DENOTES STRIDE-2 INPUT; C DENOTES CONTIGUOUS INPUT.

Payload	Op.	Base	Hoist	Batch	Combined
64 KB/S	WRITE	82.45	71.84	71.27	69.82
	READ	81.27	72.12	70.88	68.86
1 MB/S	WRITE	368.43	256.66	173.27	173.03
	READ	364.17	248.48	174.20	174.80
1 MB/C	WRITE	363.63	253.02	170.94	107.69
	READ	359.28	244.19	175.45	106.05
64 MB/C	WRITE	2838.30	2840	2894.69	2834.89
	READ	2831.36	2830	2884.74	2824.68

C. Submission and Interface Ablations

To explain the layout-dependent gains, Table I isolates validation, publication, and merging on stride-2 (S) and contiguous (C) input. Segments are 4 KB through 1 MB payloads and 256 KB above. Combined includes separate confirmation runs; 64 MB Hoist entries are estimates. Attribution uses measured variants from the same campaign.

At 1 MB/S, Batch reduces WRITE latency from 368.43 to 173.27 μs while retaining per-segment WQEs. Batch brings both layouts to roughly 170 to 175 μs, whereas Combined reaches 106 to 108 μs only for contiguous input: gaps prevent merging and retain descriptor and completion work. At 64 MB, the same 256 segments amortize submission over more payload; Batch's slight regression shows that fewer doorbells alone do not guarantee lower latency.

An eight-layout READ/WRITE control places PTO below both HIXL APIs, with sync faster than async. Asynchronous return alone therefore does not explain the gap; the artifact retains both APIs across all eight layouts.

D. Simultaneous-Direction Capacity

Table II compares initiator A, B, or both (A+B) across servers with disjoint buffers. Payload is per initiator: 1 MB uses 256 blocks of 4 KB separated by 4 KB gaps; 64/256/512 MB use 256/1,024/2,048 contiguous blocks of 256 KB. PTO uses Combined, HIXL sync pre-coalesces contiguous input, and HCCL uses BatchSendRecv with per-extent overhead for strided input. The 1/64 MB campaigns were separate; 256/512 MB use three process blocks. We compare directions within each backend.

A second initiator approximately doubles aggregate effective throughput with little change in completion time across all three backends. At 512 MB per initiator, PTO WRITE takes 22.12 ms with both active, versus 22.12/22.09 ms for A/B alone. This payload exceeds the tested kernel's merge limit, retaining 2,048 WQEs in eight publication groups. Each initiator owns its submission and completion state.

TABLE II
SINGLE- AND DUAL-INITIATOR LATENCY (MS). A/B: INITIATING SIDE; HCCL: REFERENCE.

Payload (MB)	Initiator	WRITE PTO	WRITE HIXL	WRITE HCCL	READ PTO	READ HIXL	READ HCCL
1	A	0.181	0.405	7.621	0.181	0.417	7.574
	B	0.176	0.408	7.610	0.176	0.425	7.601
	A+B	0.188	0.443	7.467	0.189	0.457	7.402
64	A	2.869	3.083	3.399	2.847	3.053	3.337
	B	2.846	3.037	3.375	2.834	3.054	3.339
	A+B	2.855	3.042	3.439	2.846	3.029	3.412
256	A	11.079	11.247	12.170	11.074	11.253	12.114
	B	11.062	11.263	12.134	11.061	11.271	12.107
	A+B	11.100	11.308	12.279	11.089	11.314	12.224
512	A	22.116	22.275	23.589	22.117	22.241	23.553
	B	22.088	22.251	23.498	22.079	22.239	23.512
	A+B	22.122	22.293	24.054	22.116	22.272	23.990

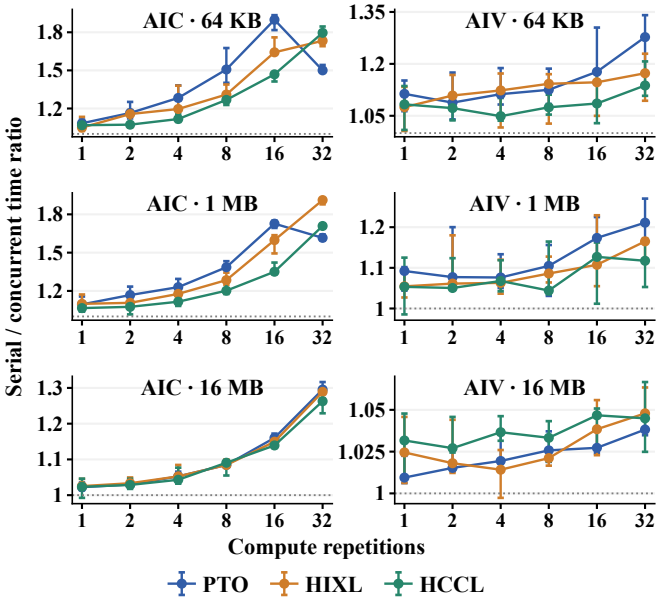


Fig. 5. AIC/AIV serial/concurrent completion-time ratios across compute workloads and transfer sizes from 64 KB to 16 MB.

E. Compute–Communication Interference

Concurrent transfers establish network progress; inference also needs computation to progress alongside them. Fig. 5 compares serial and concurrent execution of BF16 AIC GEMM or two-million-element AIV addition, repeated 1, 2, 4, 8, 16, or 32 times, alongside sixteen transfers of each size.

Serial/concurrent completion-time ratios peak near balanced work. For PTO with 64 KB transfers, increasing GEMM work first improves the ratio, then reduces it as computation dominates. Larger transfers require more compute work to

obtain a similar benefit. AIV addition yields smaller ratios than AIC GEMM, consistent with shared HBM traffic and different durations. Each backend is compared with its own serial execution. These ratios measure saved completion time, rather than the fraction of NIC activity overlapped with computation.

Additional controls check resource sensitivity and sustained protocol use. HCCL buffer controls from 16 to 512 MB show little effect on Activation/VPP batch latency. A two-hour READ/WRITE run passes payload, guard, delayed-credit, and SQ-wraparound checks and shuts down normally. Early/late median HBM samples remain stable.

V. Case Studies

We now examine how completed transfers release useful computation across pipeline stages, attention layers, and MoE experts on multiple remote peers.

A. Activation: Stage Readiness and Pipelining

Chunked pipeline parallelism (CPP) streams prefill chunks through layer stages in inference systems such as Sarathi-Serve and Qwen2.5-1M [2], [3]. Virtual pipeline parallelism (VPP) assigns multiple stages to each rank for long-context prefill [4].

This case tests activation transfer, consumption, and slot reuse. PTO runs synthetic tile computation and the ready/-credit loop in one persistent AIV kernel per endpoint; HIXL sync/async and HCCL orchestrate the same computation from the Host. The four-stage virtual pipeline revisits ranks as $0 \to 1 \to 0 \to 1$. Immediate PTO publishes ready before producing the next message; deferred PTO allows another chunk during an outstanding transfer. Timing covers all transfers and computation in each complete batch.

These interfaces can compose PyPTO-generated PTO-ISA computation [19] with cross-node activation movement and safe buffer reuse across pipeline stages.

Fig. 6 sweeps seven message sizes from 4 KB to 32 MB and 1, 2, or 4 slots. Activation batches contain 32 messages; VPP uses sixteen microbatches with 48 payload edges. Ring-wrap and withheld-credit checks confirm safe slot reuse.

Deferring ready trades producer concurrency for later consumption and credit return. With few slots, delayed credit can stall the producer; additional slots provide room for independent work. The changing policy ranking thus ties scheduling benefits to consumer readiness and slot reuse.

HCCL has lower batch completion latency in some large-message, single-slot cases, where the schedule offers little independent work. VPP ties downstream readiness to work on that rank, so publication delays affect multiple stages.

Block 0 selects the PTO schedule and HIXL API; blocks 1 to 4 evaluate those fixed choices. Held-out results support workload-specific selection, with the full calibration and validation matrix retained in the artifact.

B. KV: Transfer Granularity and Layer Readiness

Where activation releases a chunk, attention requires a layer's complete KV input. We vary migration granularity using Qwen3-8B's 72 BF16 K/V tensors, all materialized

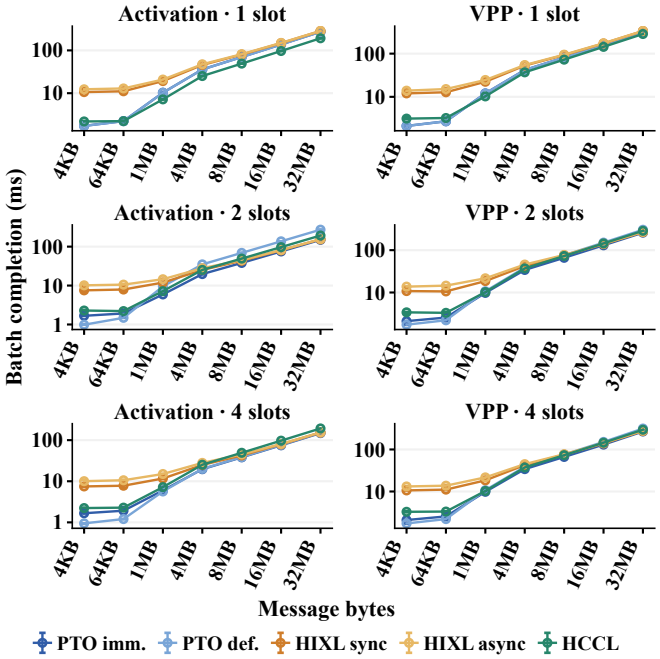


Fig. 6. Activation/VPP batch latency with different messages and slots.

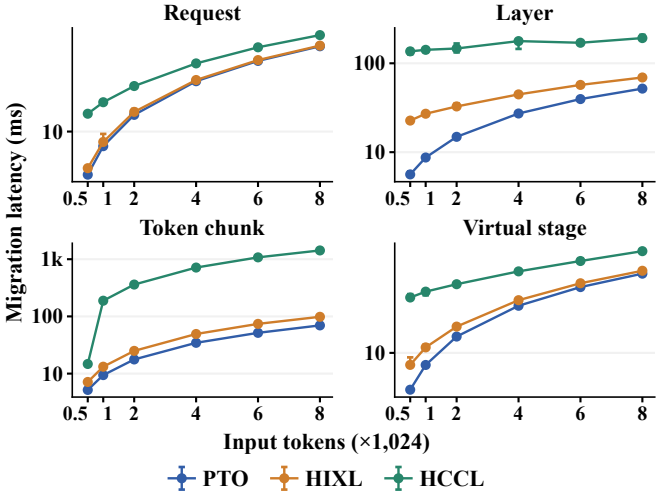


Fig. 7. KV migration latency over six input lengths and four granularities.

before transfer: one request, 36 layer groups, six virtual stages, or 512-token chunks. PTO processes extents in one AIV kernel; HIXL/HCCL use identical layouts and payloads. Each receiver layer waits for its complete input [20]. K/V, metadata, guards, and continuation tokens match the local reference without destination prefill. Timing includes planning, descriptors, uploads, receiver progress, and ACKs; HCCL includes per-extent transfers and Host readiness waits.

Fig. 7 reports migration latency for 512 to 8,192 input tokens and all four granularities. At 512 tokens, layer grouping takes 5.62, 22.67, and 136.27 ms for PTO, HIXL, and HCCL, respectively. Request grouping amortizes control; its PTO/HIXL effective throughput ratio approaches one as payload grows. At 8,192 tokens, PTO request, layer, and virtual-stage grouping takes 50.76, 51.91, and 51.17 ms, respectively, versus 69.64 ms

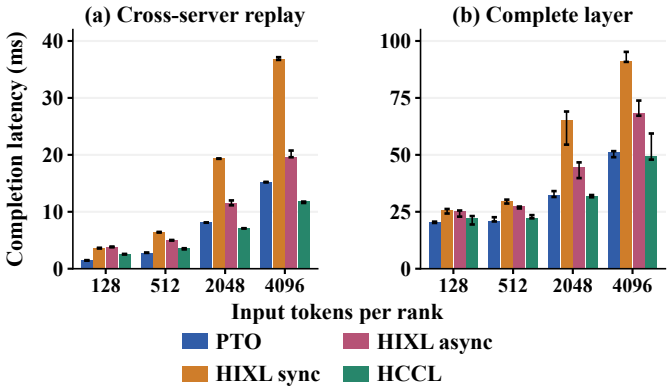


Fig. 8. EP16 natural routing: optimized PTO/HIXL and prior HCCL runs. Left: cross-server replay; right: complete layer, both with linear y axes. Bars show the median of three process p50s; error bars span their full range.

for token chunking. Virtual stages expose earlier readiness at near-request cost; token chunks add coordination costs.

For large layer/virtual-stage inputs, PTO and HIXL finish the first layer while later payload prefixes retain initial values, demonstrating progress before full-cache arrival. Token chunks still require a layer's complete K/V input. All source KV is materialized before transfer; this case measures receiver progress rather than overlap with ongoing prefill computation.

C. MoE: Sixteen-Rank Dispatch and Combine

We evaluate Qwen3-30B-A3B's first MoE layer: 128 pretrained experts, top-8, $H = 2048$, and SwiGLU dimension $I = 768$. Sixteen ranks use eight cards per server, one logical device per card, and eight experts per rank. Inputs total $16T$ tokens, with $T \in \{128, 512, 2048, 4096\}$ per rank. The router consumes RMS-normalized pretrained embeddings.

All methods share Host-scheduled BF16 AIC grouped GEMMs and FP32 combine. PTO uses one QP/AIV block per peer with 64 KiB chunks. HIXL uses 1 MiB chunks: sync serializes peers; async advances peers independently, with one outstanding chunk each. HCCL uses matched BatchSendRecv and stream synchronization. Unchanged RoCE payloads expand per expert without peer deduplication.

Full-layer timing spans routing, metadata, packing/unpacking, all-route dispatch, experts, return, combine, and completion. Cross-server replay times prepacked dispatch/return only. Both include an all-rank TCP completion barrier; setup, input generation, and validation are untimed. Replay sums phase-wise maximum rank times; the full layer uses the maximum rank interval. Three process repeats each use ten warmups and 30 timed layers. Prior HCCL runs use 2048-token arenas at $T = 512/2048$; all other plotted runs use 4096.

PTO has lower medians than HIXL across sizes (Fig. 8). At $T = 4096$, PTO, HIXL sync, HIXL async, and HCCL take 15.18, 36.87, 19.62, and 11.77 ms for replay; complete-layer times are 50.98, 91.25, 68.28, and 49.35 ms. HCCL has lower medians at the larger sizes; PTO leads at the smaller sizes.

Chunking reduces PTO's 4096-token replay from 305.66 to 15.18 ms, but raises its 512-token replay from 1.98 to 2.80 ms. A retransmission cause remains unverified. At 4096 tokens, PTO, HIXL sync, and HIXL async retain 0, 34, and 6 complete-layer samples above 100 ms out of 90, respectively.

Exact-byte, routing, guard, and BF16/FP32 oracle checks pass. With replicated oracle weights, this standalone layer establishes neither deployment capacity nor serving speedups; all records are retained.

VI. Conclusion and Future Work

We present an Ascend RDMA extension to PTO-ISA that combines device-owned submission and completion, regular-strided transfers, and ready/credit protocols. Effective READ-/WRITE throughput reaches 1.05× to 2.30× that of HIXL and 1.19× to 6.41× that of the HCCL ref. Sixteen-rank MoE shows that bounded requests reduce long waits in the tested PTO/HIXL paths, while HCCL remains competitive at larger sizes. Future work will integrate the Host-scheduled MoE case into a persistent device schedule, toward cross-node MegaMoE-style fusion [21]. This requires overlapping transfers with independent matrix tiles, preserving per-expert readiness, and returning buffer credits only after the corresponding combine inputs have been consumed.

References

[1] R. Qin *et al.*, "Mooncake: Trading more storage for less computation—a KVCache-centric architecture for serving LLM chatbot," in *FAST*, 2025.
[2] A. Agrawal *et al.*, "Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve," in *Proc. OSDI*, 2024.
[3] A. Yang *et al.*, "Qwen2.5-1M technical report," arXiv:2501.15383, 2025.
[4] Y. Shi *et al.*, "VPP: Virtual pipeline parallelism for efficient chunked prefill in long-context LLM inference," arXiv:2608.26523, 2026.
[5] Y. Zhong *et al.*, "DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving," in *Proc. OSDI*, 2024.
[6] P. Patel *et al.*, "Splitwise: Efficient generative LLM inference using phase splitting," in *Proc. ISCA*, 2024.
[7] Huawei Technologies, "NPU architecture version 220x," CANN 8.5.0 Ascend C Programming Guide, 2026.
[8] S. Kim *et al.*, "GPUnet: Networking abstractions for GPU programs," in *Proc. OSDI*, 2014.
[9] F. Daoud *et al.*, "GPUrdma: GPU-side library for high performance networking from GPU kernels," in *Proc. ROSS*, 2016.
[10] NVIDIA, "NVSHMEM: Using GPUDirect Async Kernel-Initiated Communication," NVSHMEM API documentation, 2026.
[11] C. Hwang *et al.*, "ARK: GPU-driven code execution for distributed deep learning," in *Proc. NSDI*, 2023.
[12] S. Zheng *et al.*, "TileLink: Generating efficient compute-communication overlapping kernels using tile-centric primitives," in *Proc. MLSys*, 2025.
[13] C. Hwang *et al.*, "MSCCL++: Rethinking GPU communication abstractions for AI inference," in *Proc. ASPLOS*, 2026.
[14] A. Jangda *et al.*, "Breaking the computation and communication abstraction barrier in distributed machine learning workloads," in *Proc. ASPLOS*, 2022.
[15] Huawei Technologies, "PTO-ISA: Parallel tile operation instruction set architecture," github.com/hw-native-sys/pto-isa, rev. `99ee520`, 2026.
[16] Huawei Technologies, "HIXL: Huawei xfer library," github.com/hicann/hixl, rev. `52f1829`, 2026.
[17] Huawei Technologies, "HCCL: Huawei collective communication library," gitcode.com/cann/hccl, 2026, accessed: Oct. 30, 2026.
[18] Huawei Technologies, "SHMEM: Ascend shared-memory communication library," github.com/hicann/shmem, rev. `9d938e2`, 2026.
[19] "PyPTO: A tile-based programming framework for AI accelerators," github.com/hw-native-sys/pypto, 2026, accessed: Oct. 30, 2026.
[20] Huawei Technologies, "Torch-NPU stream handle and host task queue implementation," github.com/Ascend/pytorch, 2026.
[21] C. Zhao *et al.*, "DeepGEMM: Clean and efficient BLAS kernel library on GPU," Mega MoE documentation, github.com/deepseek-ai/DeepGEMM, 2025.